

Path C-min Completion Report

仓库: `supply-chain-twin-D3_pathc`

状态基线: 本报告所有结论均基于仓库当前实际文件与测试运行结果 (`env -u PYTHONHASHSEED python3 -m pytest` → **1259 passed, 0 failed, 0 skipped**)。

1. Executive Summary

Path C-min 是什么。 在 Path B (V2 agents + compiled graph + 20 oracle cases + 事件循环 + outcome store) 冻结的基础上, 用 *加法式扩展* 构建一条 policy-layer-only 的自适应半自治闭环。learning 只通过 adaptive policy gate 进入系统, 决策不污染 Research Core, Path B 主线只做了两处严格受限的 additive 修改。做到了什么。

- Phase 0: 把 Path B 当前行为冻结为 byte-exact fixtures (`demo_stream` 快照 + 20×3 oracle matrix 快照), 引入 `learning / adaptive / session / outcome_persistence` 四个 Path C 子包 + `PATH_C_BOUNDARY.md` + `PATH_C_SCHEMA_REGISTRY.md` + AST topology guard。
- Phase 1: EpisodicMemory (append-only、确定性排序、query/summarize/snapshot)、JSONL 持久化、全 64-char SHA-256 `session_id`、`event_loop_c.run_session wrapper`、save/load/replay byte-identical。
- Phase 2: `decide_policy_adaptive` + AdaptivePolicyGateConfig + cold-start guard (`min_records_for_shift` 参数化) + `validate_governance_meta preflight` + `execute_action(require_metadata=True)` Path C main path; `PATH_C_COLD` / `PATH_C_WARM` 模式; memory 首次真正影响 route。
- Phase 3: KPI 计算器 (Roadmap §3.D source-of-truth 表) + Session compare (`kpi_matrix / deltas / diverged_events / thesis_claim_support`) + 一键 eval harness + 决定性 thesis markdown。

没有做到的 (刻意留出)。所有 Path C-min 之外的能力: replan、event correlator、cross-session cumulative memory、agent-visible episodic memory experiment、React / 任何 UI。

完成范围。严格到 Phase 0 / 1 / 2 / 3, 不包含 backlog。

2. Scope and Boundary

Scope (做什么)。 Path C-min 的唯一任务是：给 Path B 加一层 *policy-gate-level* 的可审计自适应 overlay，让 memory 能在 policy 决策这一层按结构化规则触发 one-level risk upgrade 或 cold-start fallback，并产出可比对的 session artifacts / compare report / thesis report。

非协商原则 (代码中被测试强制)。

- **Deterministic-first.** 禁止 `datetime.now()` / `utcnow()`、`time.time()`、`uuid.uuid4()`、裸 `datetime(...)`。强制 `sort_keys` JSON、确定性 dict 迭代、`round(x,4)` 只出现在 serialization/reporting 边界。由 `tests/test_path_c_import_topology.py` AST 扫描 `src/learning/`、`src/adaptive/`、`src/session/`、`src/outcome_persistence/`、`src/event_loop_c.py` 并在同文件自带正/负 self-test。
- **Truth boundary explicit.** `governance_output["risk_level"]`、`recommended_action`、`_governance_meta["recommended_candidate_type"]` 是 Tier-1 governance truth，Path C 只读；adaptive overlay truth 仅存在于 `SessionEventRecord.effective_decision` 与 `adaptive_adjustment`。两者在每条 `SessionEventRecord` 上并列出现，绝不合并为一个值（由 `test_adaptive_audit_trail.py`、`test_session_compare.py` 校验）。
- **Layer separation explicit.** Path C 代码不 mutate Path B 原始记录；`SessionEventRecord.baseline_event_result` 以 dict-copy 嵌入 Path B `event_result`，byte-equal 于直接 `run_event_loop` 的输出（由 `test_baseline_event_result_still_byte_equal_in_warm_mode` + `test_phase0_demo_stream_still_byte_identical_via_event_loop_c` 校验）。
- **Policy-only learning injection.** V2 agents (`operations_agent.py` / `cost_agent.py` / `governance_agent.py` 8-field schema) 零修改；`scenario_context["outcome_summary"]` 仍是 Path B 自己的 summary；memory 仅通过 `decide_policy_adaptive(..., memory=...)` 这一条参数通道进入系统。
- **No new mutation style.** `EpisodicMemory` / `JSONLStore` 均为 append-only；`SessionArtifact` 从外部视角不可变；没有在 `twin_state.py` 新增 mutation helper；Phase 2 的 Path C main path 执行仍走原有 `execute_action` 的 deep-copy / `apply_action` / `compute_total_cost` 路径。
- **Bounded action set.** `AdaptivePolicyAdjustment.adjustment_type` $\in \{\text{UPGRADE_ONE_LEVEL}, \text{NO_ADJUSTMENT}, \text{COLD_START_FALLBACK}\}$ ；never DOWNGRADE；升级最多一级 (`LOW`→`MEDIUM` / `MEDIUM`→`HIGH`，绝不 `LOW`→`HIGH`)。

Research Core 未被污染的证据链。

1. `tests/test_path_c_import_topology.py` 对所有 Path C 目录做 AST 扫描，强制禁止 `import evaluation` / `import action_code_mapper`、`open("data/cases/...")`、`os.path.join(..., "data", ..., "cases", ...)`、`Path("data/cases/...")`、`Path(...)` / `"data"` / `"cases"` 链、`.open()/read_text()` 指向 `data/cases` 的接收器。同文件自带 13 个正样本 + 4 个负样本 self-test。
2. Phase 0 `tests/test_d3_baseline_v1_oracle_matrix.py` 固定 20 cases × 3 supervisor modes 经 `compile_graph()` 的 evaluation 结果为 byte-exact fixture；每个后续 phase 仍然 green → Research Core 评估语义未漂移。
3. `evaluation.py`、`action_code_mapper.py`、`data/cases/*.json` 无任何修改（仓库工作树在这三项上零 diff）。

3. Phase-by-Phase Delivery

Phase 0 — Contract freeze + Baseline snapshot + Boundary hardening

- 目标。锁定 Path B 当前行为，生成可复现 fixture；为 Path C 建立 schema 骨架与 CI-enforced boundary。
- 实际实现。
 - Path C schema 骨架（仅类，无运行时逻辑）：
 - [src/learning/memory_schema.py](#) — MemoryRecord、MemoryQuery、MemorySummary (v1.0)。
 - [src/adaptive/adaptive_schema.py](#) — AdaptivePolicyAdjustment、AdjustmentRuleSpec (v1.0)。
 - [src/session/session_schema.py](#) — GovernanceTruthRef、EffectiveDecisionRef、SessionEventRecord、SessionConfig、SessionKPIs、SessionArtifact (v1.0)。
 - Path B additive-only: outcome_store.summary() 多加顶层 "schema_version": "1.0"；event_loop.run_event_loop 每个 event_result 多加顶层 "schema_version": "1.0"。
 - 确定性硬化 (Phase 0 correction pass): src/agents/governance_agent.py 里 rationale_trace 的 list({ref.get("source_doc",...) for ref in ...}) set comprehension 替换为保序 first-seen 去重，消除跨进程 hash-seed-dependent 漂移。这是整个 Path C-min 对 Path B agents 的 唯一修改，且不改变业务语义（输出仍是同 3 个 source docs）。
 - Boundary & registry: [PATH_C_BOUNDARY.md](#) + [PATH_C_SCHEMA_REGISTRY.md](#)。
 - Fixtures: [tests/fixtures/d3_baseline_v1_demo_stream.json](#)、[tests/fixtures/d3_baseline_v1_oracle_matrix.json](#)、[tests/fixtures/path_c_schema_samples_1_0.json](#)。
 - 关键测试。test_d3_baseline_v1_snapshot.py (byte-exact demo stream + within-process + fresh-subprocess hash 一致)、test_d3_baseline_v1_oracle_matrix.py (20×3 byte-exact)、test_path_c_schema_frozen.py (字段集 / default / roundtrip)、test_path_c_import_topology.py (AST guard + 自检样本)、test_outcome_summary_version.py。
 - 达到的能力。决定性 baseline + 不可漂移的 schema + CI-enforced boundary。
 - 这一 phase 没做什么。没有任何 memory 写入、adaptive 决策、session 组装、持久化或 CLI。

Phase 1 — Episodic Memory Core + Session Skeleton + Persistence

- 目标。建起 Path C 的 memory/session 衬底；memory 只写不读，decision 与 Path B 完全一致。
- 实际实现。
 - [src/learning/episodic_memory.py](#) — EpisodicMemory append-only, _canonical_order 按 (event_timestamp, event_id) 排序；query 支持 event_type/action_type/final_route/recent_n；snapshot() 返回 {"records": [...], "schema_version": "1.0"}；无 delete/update/replace surface（由测试反向验证）。
 - [src/learning/memory_summarizer.py](#) — 纯函数 summarize_records(records, query, *, threshold)；zero-denominator 返回 None；query_signature 是 keys-sorted 字符串。

- [src/outcome_persistence/jsonl_store.py](#) — append-only JSONL, 每行 `json.dumps(..., sort_keys=True, separators=(",", ":"))`; `read_all()` 容忍末尾截断行 (crash tolerance)。
- [src/session/digests.py](#) — `canonical_json` / `sha256_hex` / `digest_config_dict` / `digest_event_stream` / `digest_memory_records` / `compute_session_id` (`sha256(f"{{seed}}|{{config_digest}}|{{event_stream_digest}}|{{initial_memory_digest}}"`) 完整 64-char hex) / `memory_record_id`。
- [src/session/session_manager.py](#) — `save_session` / `load_session` / `replay_session_bytes`; on-disk 布局 `session_artifact.json` + `memory.jsonl`。
- [src/event_loop_c.py](#) — `wrap_run_event_loop` 为 baseline shadow; 每 event 追加 1 条 `MemoryRecord` 并构造 `SessionEventRecord` (baseline_event_result 以 dict-copy 嵌入)。
- [scripts/run_session.py](#) — 最小 CLI。
- 关键测试。 `test_episodic_memory.py` (12)、`test_memory_summarizer.py` (13)、`test_jsonl_store.py` (7, 包含 crash-tolerance)、`test_session_manager.py` (5)、`test_event_loop_c_wrapping.py` (9)、`test_replay_byte_identical.py` (3)、`test_determinism_stress.py` (3, fresh-subprocess)、`test_phase0_baseline_regression_phase1.py` (2)。
- 达到的能力。一条 `event_loop_c.run_session(seed, mode)` 调用就能产出 deterministic `SessionArtifact`, `save/load` 字节同一, 跨进程 hash 同一。
- 这一 phase 没做什么。memory 没有任何消费者; `PATH_C_COLD` 与 `BASELINE_STATIC` 在 Phase 1 终态决策层完全等价。

Phase 2 — Adaptive Policy Gate + Cold-Start Guard + Execution Contract Hardening

- 目标。让 memory 首次真正影响 behavior, 但只允许在 policy gate 层; Path C 主路径强制 metadata, 禁止文本 fallback。
- 实际实现。
 - [src/adaptive/adaptive_policy_config.py](#) — `AdaptivePolicyGateConfig` (`canonical owner`: `min_records_for_shift`, 默认 3)、`EventContext`、`POOR_SLA_UPGRADE_RULE_ID`、`DEFAULT_RULES = (_poor_sla_upgrade_rule,)`、`KNOWN_RULE_IDS`。
 - [src/adaptive/cold_start.py](#) — `is_cold_start(matched, threshold)`、`build_cold_start_adjustment(...)`。
 - [src/adaptive/preflight.py](#) — `validate_governance_meta` + `GovernanceMetaPreflightError`; 接受四个 operational literal; 显式拒 `APPROVE/VERIFY/OVERRIDE` 与 `AI/ALT1/ALT2`。
 - [src/adaptive/adaptive_policy_gate.py](#) — `decide_policy_adaptive(gov, ctx, memory, config) -> (PolicyDecision, AdaptivePolicyAdjustment | None)`。算法: 按 event type 构造 `MemoryQuery` → `summarize` → `cold-start` 先决 → 规则序列 `first-fire-wins` → `UPGRADE_ONE_LEVEL` 时在 `governance_output` 的本地副本上改 risk 并重跑 `decide_policy`, 永不修改原 `governance_output`。
 - [src/execution_adapters.py](#) — 唯一允许触碰的 Path B 文件。`_parse_operational_type` 与 `execute_action` 新增 `require_metadata: bool = False`。默认保持 Path B 回归; `True` 时 `metadata` 缺失/无效直接抛 `GovernanceActionParseError`, `text fallback` 不可达。
 - [src/event_loop_c.py](#) — 增加 `_build_path_c_main_records`: 对每 event 运行内部 reasoning slice (从 `event_loop` 导入 `_run_reasoning_slice / _twin_state_to_agent_dict` / `load_baseline_twin_state`), 再调 `adaptive gate`, 再 `preflight`, 再 `execute_action(require_metadata=True)`, 并维持独立的 Path C live state + `OutcomeStore`。Path B shadow 始终用完整 `run_event_loop` 单独跑一次产出 `baseline_event_result`。

- **关键测试。** `test_adaptive_policy_gate.py` (8)、`test_adjustment_rules.py` (14, 含 no-downgrade 跨 risk×sla 矩阵扫描)、`test_cold_start_behavior.py` (4, 参数化 N=3 / N=5)、`test_governance_meta_preflight.py` (12)、`test_execution_metadata_mandatory.py` (7, 含 Path B 文本 fallback 回归)、`test_path_c_vs_baseline_divergence.py` (3, R1 验证)、`test_adaptive_audit_trail.py` (4, 结构化可解释性)。
- **达到的能力。** seeded poor-SLA memory 下, `PATH_C_WARM` 对 `CARRIER_DELAY` 事件从 LOW 升到 MEDIUM, route 从 `AUTO_EXECUTE` 改 `HUMAN_REQUIRED`; `baseline_event_result` 仍然 `byte-equal` 于直接 `run_event_loop`; `preflight_failed` 只在 overlay 出现。
- **这一 phase 没做什么。** 没有 KPI / compare / report / UI; 没有 DOWNGRADE; 没有 `event_type×action_type` 二维阈值; 没改任何 V2 agent。

Phase 3 — Session Eval Harness + A/B Compare + Thesis-Aligned Reports

- **目标。** 外部验证工具链: 一条命令跑 `baseline_static / path_c_cold / path_c_warm`, 产出结构化 compare + thesis markdown。
- **实际实现。**
 - [src/session/kpi_calculator.py](#) — `compute_phase_kpis / compute_session_kpi_report / segment_records` (cold = events 1..N, warm = events N+1..end) / `round_kpi_report`。每个字段只有一个 canonical reader (`_effective_execution_status / _effective_final_route / _effective_action_taken / _baseline_cost_incurred / _baseline_sla_preserved`)。
 - [src/session/session_compare.py](#) — `build_compare_report(artifacts, baseline_mode, min_records_for_shift)`; 输出 `schema_version / sessions_compared / baseline_mode / min_records_for_shift / per_session_kpi_report / kpi_matrix / deltas / diverged_events / thesis_claim_support`; `render_thesis_markdown` 是 pure function。
 - [scripts/session_eval_harness.py](#) — CLI `--seed / --events-source / --out-dir / --min-records-for-shift / --warm-memory {synthetic,empty}` / `--n-synthetic-records`。
 - [scripts/generate_thesis_report.py](#) — 独立 markdown renderer。
 - [docs/path_c_thesis_alignment.md](#) — thesis alignment 文档, 明写 honesty rule。
- **关键测试。** `test_kpi_calculator.py` (14, 含 hand-constructed 精确数值 / zero-denom / 参数化 segmentation / 只在 serialization 舍入)、`test_session_compare.py` (7, 含 compare-to-self 性质 + 已知 divergence 连到 adjustment + 同输入纯函数渲染)、`test_eval_harness_reproducible.py` (3, in-process + fresh-subprocess + cold-run identity claim)、`test_thesis_report_generation.py` (8)。
- **达到的能力。** 一条 `python scripts/session_eval_harness.py --seed 42 --out-dir ...` 产出 3 个 session artifact + 3 个 JSONL + `compare_report.json` + `thesis_report.md`; 3 个独立 fresh Python 进程产出的 compare/thesis bytes sha256 完全一致 (`compare=03ed68a4... thesis=2c54bfbb...`)。
- **这一 phase 没做什么。** 没有 UI; 没有任何 runtime decision logic; 没有 bootstrap / Monte Carlo 等非决定性统计; 没有 `supports_claim` 的 post-tuning。

4. Architecture Summary

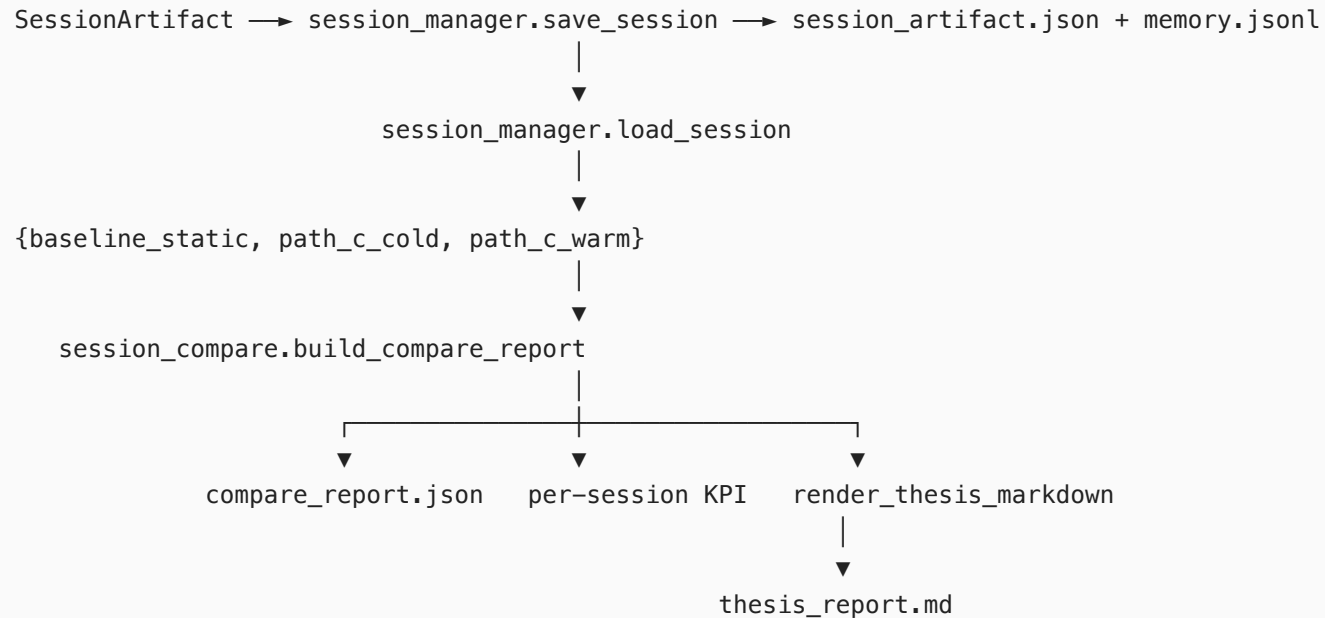
三层记录 (roadmap §0.1)。

- **Tier 1 — Path B raw output (冻结)**。 `EventPayload` / `ExecutionOutcome` / `PolicyDecision` / `GovernanceOutput` (8-field) / `_governance_meta` (3-field) / Path B `event_result` (9 keys + `schema_version=1.0`) / `outcome_summary` (11 fields + `schema_version=1.0`)。Phase 0 仅加 `schema_version` key; 后续 phase 再未动。
- **Tier 2 — Path C session overlay (新)**。 `SessionEventRecord` 包一个 未修改的 Path B `event_result` 作 `baseline_event_result` 字段, 并外挂 `mode` / `policy_route_source` / `adaptive_adjustment` / `governance_truth` / `effective_decision` / `memory_record_id` / `notes`。
- **Tier 3 — Path C session artifact (新)**。 `SessionArtifact` = `session_id` + `config` + `event_records` + `memory_snapshot` + `kpis` + `schema_versions` + `notes`。

Runtime 组装。



Persistence / Report.



层间隔离。Research Core (evaluation.py / action_code_mapper.py / data/cases/*.json) 与 V2 agents 完全不被 Path C 代码 import 或间接触达 (AST-guard 强制)。Tier 1 的读只发生在构造 Tier 2 时的 dict-copy; Tier 2/3 的写只来自 Tier 2/3 的构造器。decide_policy_adaptive 只读 governance_output , UPGRADE_ONE_LEVEL 时改写的是 dict(governance_output) 的 本地副本 (测试 test_governance_output_is_not_mutated 保证)。

5. Determinism and Auditability

- **session_id**. 64-char 全 SHA-256 hex, 输入为 `f"{seed}|{config_digest}|{event_stream_digest}|{initial_memory_digest}"`; 无 wall-clock 输入。 `test_same_inputs_same_id + test_different_{seed,mode}_changes_id + test_same_seed_same_artifact_across_fresh_processes` 同时锁死。
- **Digests**. 所有 digest 走 `session.digests.canonical_json (sort_keys + compact separators + 自定义 encoder: pydantic 走 model_dump, datetime 走 isoformat, enum 走 .value, set 走 sorted(list(...)))`。
- **JSONL**. 每行 canonical JSON; append 后 fsync; read_all 跳过末尾截断行 (由 `test_crash_tolerance_preserves_prior_complete_lines` 反向验证)。
- **Replay byte-identity**. `canonical_json(load(save(A)).model_dump()) == canonical_json(A.model_dump())` (`test_artifact_roundtrip_byte_identical`), 同时 JSONL 每行与 `canonical_json(record)` 一致。
- **Fresh-process determinism**. 三处 3 子进程实测:
 - Phase 0 demo stream (`test_demo_stream_output_deterministic_across_fresh_processes`)。

- Phase 1 `run_session` (`test_same_seed_same_artifact_across_fresh_processes`)。
 - Phase 3 `eval harness` (`test_harness_fresh_subprocess_deterministic`, `compare` + `thesis` 双哈希匹配)。
三个场景全部 pass; Phase 0 的不依赖 `PYTHONHASHSEED` 通过源码级 `hardening` 达成, 不靠测试 `harness` 钉死 `seed`。
 - **Deterministic hardening** 落点。仅一处 Path B: `src/agents/governance_agent.py` 的 `rationale_trace set-comprehension` → 保序 dedup, 细节见 `PATH_C_BOUNDARY.md §8`。
 - **Rounding boundary**. 搜索可知 `round(x, 4)` 仅出现在 `kpi_calculator._round_phase_kpis`、`session_compare._round_*`、`thesis_report` 渲染路径; 决策逻辑 (`decide_policy_adaptive / rules / cold-start / preflight`) 全程 full precision。
 - **Explainability**. `AdaptivePolicyAdjustment` 每次 fire 时强制携带:
 - `rule_id` (来自 `KNOWN_RULE_IDS` 注册表);
 - `query_signature` (等于 `MemorySummary.query_signature`, 即 `keys-sorted MemoryQuery` 序列化);
 - `memory_evidence` (`UPGRADE_ONE_LEVEL` 走 `{matched_records, sla_preservation_rate, threshold_rate}`; `COLD_START_FALLBACK` 走 `{matched_records, min_records_for_shift}`);
 - `pre/post_adjustment_risk` 满足「`pre != post iff UPGRADE_ONE_LEVEL`」。可 replay 性由 `test_replay_same_memory_slice_same_fire_outcome` + `test_numerical_evidence_consistent_with_memory_slice` 校验。测试中不使用 `length-based` 或 `string fuzzy-match` 判据。
-

6. Test and Verification Summary

- **Baseline freeze**. `test_d3_baseline_v1_snapshot.py` (`run_event_loop(demo_stream)` `byte-exact` + `fresh-subprocess hash` 一致)、`test_d3_baseline_v1_oracle_matrix.py` (`20 cases × 3 supervisor modes via compile_graph()` `byte-exact`)、`test_phase0_baseline_regression_phase1.py`、`test_phase0_demo_stream_still_byte_identical_via_event_loop_c`。
- **Schema freeze**. `test_path_c_schema_frozen.py` (`11 Path C schema` 的字段集 + `default` + `roundtrip`)、`test_outcome_summary_version.py` (`Path B additive schema_version=1.0` 存在且不变)。
- **Topology guard**. `test_path_c_import_topology.py` 扫描 Path C 目录 + `13 positive` + `4 negative` 自检样本。
- **Cold-start parameterization**. `test_cold_start_behavior.py` 覆盖 $N \in \{1, 3, 5, 10\}$ 的 `is_cold_start` boundary, $N \in \{3, 5\}$ 两组参数化: `< N` 必产 `COLD_START_FALLBACK`; `== N` 是第一个可触发 `sample`; `PATH_C_COLD` 前 `N` 个事件的 `policy_route_source` $\in \{\text{baseline_static, cold_start_fallback}\}$ 。
- **Adaptive divergence (R1)**. `test_path_c_vs_baseline_divergence.py` 用 `10` 条 `poor-SLA CARRIER_DELAY` 种子记忆, 跑 `BASELINE_STATIC` vs `PATH_C_WARM`, 断言至少一事件 `effective_decision.final_route` 不同且 `adaptive_adjustment` `is not None`; 同时断言 `baseline_event_result` 与直接 `run_event_loop` `byte-equal` 且 `preflight_failed` 不写入 `baseline`。

- **Metadata-mandatory path.** `test_execution_metadata_mandatory.py`: `require_metadata=True` + `metadata` 缺失 → `GovernanceActionParseError`; `True` + 合法 `metadata` → `metadata` 路径成功; `False` + 合法 `text prefix` → `fallback` 仍可用 (Path B 回归不变)。
- **Preflight.** `test_governance_meta_preflight.py` (12) 覆盖 `dict-type` / 空串 / 非法 `token` / 未知 `operational token` / 缺少 `recommended_candidate_id`。
- **Structural explainability.** `test_adaptive_audit_trail.py` 检查 `rule_id` ∈ `KNOWN_RULE_IDS`、`query_signature` 非空、`memory_evidence` 至少含 `{matched_records, auto_execute_success_rate, sla_preservation_rate}` 之一且数值与 `memory slice` 一致、`pre != post` iff `UPGRADE_ONE_LEVEL`、`replay` 同 `slice` → 同 `fire/no-fire`。
- **KPI source-of-truth.** `test_kpi_calculator.py` 14 个测试手工构造记录精确比对 `sla_preservation_rate` / `avg_cost_per_event` / `calibrated_autonomy_score` / `human_escalation_rate` / `auto_execute_rate` / `known_outcome_coverage`; `zero-denom` 全返回 `None`; `rounding` 仅在 `_round_phase_kpis` 应用。
- **Compare / report reproducibility.** `test_session_compare.py`: `compare-to-self` 时 `diverged_events == []`, 所有 `delta` `0` 或 `None`; 已知 `divergence` 会被 `adjustment_ref` 对应到具体 `AdaptivePolicyAdjustment`; `render_thesis_markdown` 纯函数。`test_eval_harness_reproducible.py`: `in-process` 两次 + `fresh-subprocess` 两次产出 `bytes` 完全一致; 冷启动 `identity claim` 用 `min_records_for_shift=100` 反向验证整段 `cold` 时 `baseline↔cold` 零 `divergence`。
- 汇总。仓库当前 `pytest` 全量 **1259 passed, 0 failed, 0 skipped** (`env -u PYTHONHASHSEED python3 -m pytest`)。

7. Known Deviations / Tradeoffs

只列 仓库当前实际 存在的设计选择。

1. **Phase 1/2/3 helper** 没写进 **Phase 0 frozen schema** 文件。Roadmap §1.C / §2.C 字面写着在 `memory_schema.py` / `session_schema.py` / `adaptive_schema.py` 新增 `"serialization / factory / validator helpers"`; 仓库把它们放到了 `memory_summarizer.py`、`session_manager.py`、`digests.py`、`adaptive_policy_config.py` 这些并列新文件里。动机: 维持 Phase 0 的 `test_path_c_schema_frozen.py` 字段集严格锁死。功能等价, `review surface` 更干净。设计选择。
2. **GovernanceTruthRef.recommended_candidate_type** 的 **Phase 1 shim** 在 **Phase 2** 仍作为 **fallback** 保留。Path C main path 优先读 `_governance_meta.recommended_candidate_type` (canonical); `malformed` 时退到 `_recommended_candidate_type_shim` (解析 `recommended_action` 前缀为四 `operational literal` 之一)。这是结构化 `metadata-only` 字段, 从不参与任何 `decision path`, 仅写入 `SessionEventRecord.governance_truth` 做审计。设计选择, 不是技术债。
3. **SessionArtifact.kpis** (**Phase 0 frozen SessionKPIs**) 在 **Phase 3** 仍然只填 `events_observed`。Phase 3 全套 KPIs (`calibrated_autonomy_score` 等 6 项) 放在 `compare report` 的 `per_session_kpi_report` 与 `kpi_matrix` 里, 没有塞回 `artifact`。理由: `SessionKPIs` 字段集在 Phase 0 注册表为 `v1.0`, Phase 3 不打算 `bump` 版本。设计选择。
4. **BASELINE_STATIC** 模式完全不经 **Path C main path**。`require_metadata=True` 与 `preflight` 仅对 `PATH_C_COLD` / `PATH_C_WARM` 启用。`BASELINE_STATIC` 继续走 Path B 默认 `require_metadata=False` (含 `text fallback`)。这是 Roadmap §2.B 的显式语义——要测 Path C hardening, 需切到 Path C 模式。设计选

择。

5. **Eval harness** 默认用一个确定性合成 **seed memory** (10 条 **poor-SLA CARRIER_DELAY**) 给 **PATH_C_WARM**。6 事件的 demo stream 没有足够的同类历史能让 warm 期自然触发规则；没有 seed 的话 thesis 对比没有信号。harness `--warm-memory empty` 提供 zero-memory 回退；setup 在 [docs/path_c_thesis_alignment.md](#) 有明文声明。设计选择，不是隐藏实验条件。
6. 默认 **CLI smoke** 下 `thesis_claim_support.supports_claim = False`。原因是当前默认 seed 下 warm 升级事件路由走 **HUMAN_REQUIRED**，剩余 **AUTO_EXECUTE** 事件的 `avg_cost_per_event` 升高，而 SLA 与 CAS 没有同步收益。Roadmap §3.F rollback 条款明确要求如实上报。后续可优化点：扩张 **DEFAULT_RULES** 或调整 seed 覆盖的 event type，使得 thesis claim 更有机会被支持——但此轮禁止 post-tune，所以留作 post-Phase-3 scope。
7. **Path C main path** 通过从 `event_loop import _run_reasoning_slice / _twin_state_to_agent_dict` (下划线前缀) 调用 **Path B** 内部。这是 Python 下划线惯例下的“半内部”依赖。event_loop.py 并未修改。如果未来 Path B 重构这两个私有函数，Path C main path 需对应更新。后续可优化点：将它们提升为 Path B 公开 API (需 Path B owner 授权)。
8. 仓库工作树中 `src/graph.py / src/twin_state.py` 对原始 commit 4fae044 的 diff 属于 **Path C-min** 开工前就已存在的 **Path B baseline** 工作 (`apply_event_patch / apply_outcome`、**supervisor** 注释修正)，**Path C-min** 没有触碰它们。这一事实已在 Phase 0 correction pass 的 cleanup report 中写明。设计选择 (历史遗留，非 **Path C-min** 修改)。

8. Backlog Explicitly Out of Scope

- **B1. Replan loop.** 当实际成本显著偏离预期时重新触发推理。为什么不进入主线：需要先定义 `expected_cost_range` 契约并且只能来源于结构化 `cost_output`，这要求新一轮 boundary 讨论，超出 Path C-min 的 policy-only learning injection 范围。
- **B2. Event correlator.** 检测近邻事件的共现模式、发出复合告警。为什么不进入主线：会要么扩展 `EventType` 枚举 (打破 Path B 的封闭集合假设)，要么把关联以 sideband `correlation_context` 表示，两种都需独立 owner 决议。
- **B3. Cross-session cumulative memory.** 多 session merge、warm-start policy、dedupe 规则。为什么不进入主线：需要显式的 merge/append/dedupe 语义与 **PATH_C_WARM** prior-memory loading 机制，不是 Path C-min 要解决的问题；Phase 1 的 session-local memory 已经完成“memory 可写可查”的 minimum。
- **B4. Agent-visible episodic memory experiment.** 让 V2 agents 直接读 memory summaries。为什么不进入主线：Path C-min 明确把 memory 消费者限制在 policy gate 这一层；开这一口子会违反 policy-only learning injection 的非协商原则，必须以独立 feature branch + 单独 boundary doc 推进。
- **B5. React / 前端 / dashboard.** 事件时间线 UI、决策审阅工作区、session 对比可视化。为什么不进入主线：产品表面工作，应消费 已冻结 的 Path C-min 契约，前端不允许新增任何决策逻辑；必须另起 branch 与 owner scope。

9. Final Status

`PATH_C_MIN_COMPLETE_WITH_NOTED_TRADEOFFS`

完成事实。 四个 phase 全部交付；仓库当前测试套件 **1259 passed / 0 failed / 0 skipped**；Path B 唯一修改范围严格限于 `execution_adapters.py` 的 `require_metadata` 加法 + `governance_agent.py` 的确定性排序硬化；Research Core 零污染（AST-guard + 20×3 oracle matrix 跨 phase 锁死）；所有 non-negotiable principles（deterministic-first / truth boundary / layer separation / policy-only learning injection / no new mutation style / bounded action set）在代码与测试中均有强制支撑。

保留的 tradeoff。 第 7 节列出的 8 条全部属于 *已知且自觉* 的设计选择或 *后续可优化点*，不阻塞 Path C-min 本身的 completeness——尤其第 6 条（默认 smoke 下 `supports_claim=False`）是诚实性条款的正确呈现而非 bug。

可验证入口（reviewer 自检）。

- `env -u PYTHONHASHSEED python3 -m pytest → 1259 passed。`
- `env -u PYTHONHASHSEED python3 scripts/session_eval_harness.py --seed 42 --events-source demo_stream --out-dir /tmp/pathc_review → 3 artifact 目录 + compare_report.json + thesis_report.md`；二次运行与任何 fresh Python 进程复跑 bytes 一致。
- 仓库 `git diff HEAD -- src/` 仅显示 `execution_adapters.py` 新增 `require_metadata` 关键参数、`governance_agent.py` 的 `set-comprehension` → 保序 dedup、以及 Path B 开工前就存在的 `graph.py` / `twin_state.py` 遗留 diff；其他 Path B 文件零改动。

报告结束。